UTM
UNIVERSITI TEKNOLOGI MALAYSIA

SECJ 3303 – INTERNET PROGRAMMING

TOPIC 8 – USING RELATIONAL DATABASE

innovative • entrepreneurial • global



UTM JOHOR BAHRU

OBJECTIVES

Knowledge

- Understand the Role of Databases in Web Applications
- Overview of database connection methods.
- Traditional JDBC connection approach.
- Using non-PreparedStatement vs PreparedStatement for more secure database queries.
- Performing CRUD operations with standard JDBC.
- CRUD operations using PreparedStatement.
- Overview of Spring JDBC Template.
- Performing CRUD operations with Spring JDBC Template.
- Configure Hibernate in a web application.
- Map entities to database tables using annotations.
- Perform CRUD operations using Hibernate.

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

OBJECTIVES

Applied

- Configure JDBC database connection.
- Implement basic CRUD operations.
- Apply parameterized queries for security.
- Set up data source and Spring JDBC Template.
- Set up Hibernate configurations.
- Handle autogenerated keys for identity columns.
- Implement CRUD operations with Hibernate.

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Database in Web Application

In modern web development, the integration of databases is a cornerstone for managing data and enabling dynamic, data-driven applications

Importance of Databases in Web Development

- Data Storage and Retrieval: Databases serve as repositories for storing and retrieving data. They enable applications to persistently store information and retrieve it as needed for dynamic content presentation.

- Dynamic Content: Web applications often rely on databases to dynamically generate content based on user interactions, preferences, and real-time updates.

- User Authentication: Databases are integral for managing user accounts and authentication. User credentials and profiles are securely stored, allowing for secure access to applications.

UTM UNIVERSITI TEKNOLOGI MALAYSIA

Database in Web Application (Cont...)

Understanding the dynamic role of databases is essential for web developers. Whether opting for traditional relational databases or exploring cloud-based solutions like Firebase, developers have a range of tools to choose from based on the specific needs of their applications

Here we will cover 3 different approaches

1. **Using JDBC** - (Java Database Connectivity), which is a standard Java API for database-independent connectivity between the Java programming language and a wide range of databases.
2. **Using Spring JDBC Template -** focuses on the Spring JDBC Template, a higher-level abstraction that simplifies database operations within the Spring framework.
3. **Using Hibernate -** an ORM framework.

UTM UNIVERSITI TEKNOLOGI MALAYSIA

Using Relational Database in Web Application

Part 1:

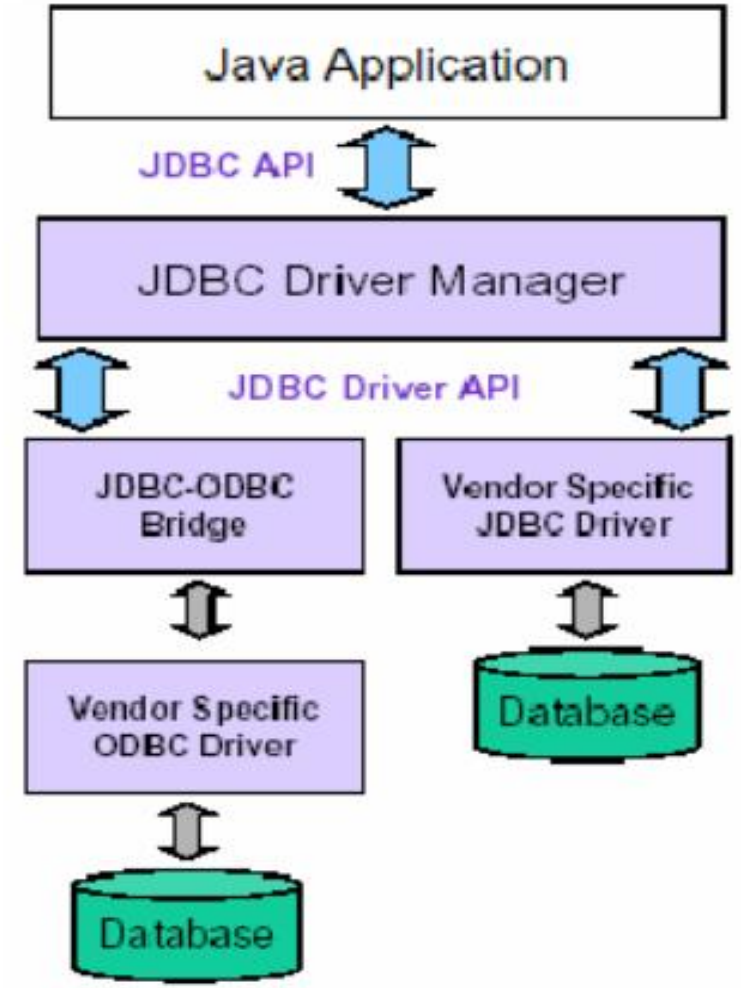
Using JDBC - (Java Database Connectivity), which is a standard Java API for database-independent connectivity between the Java programming language and a wide range of databases.

JDBC serves as a bridge between Java applications and relational databases, enabling seamless communication, data retrieval, and manipulation.

JDBC simplifies the process of working with databases in Java programs and provides a standardized way to interact with different database management systems

UTM UNIVERSITI TEKNOLOGI MALAYSIA

JDBC General Architecture



Preparations

Prep 1: Use your existing spring web project
(recent spring web project from topic 8 - Spring Web)
Add dependency to project's pom.xml

```
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.23</version>
</dependency>
```

Prep 2: mysql database, with name="springdb",
2 tables : i. Customer (id, name, address, contactNum, email)
ii. Product (id, name, price, quantity)

Prep 3 : insert several records to table Customer and Product

UTM UNIVERSITI TEKNOLOGI MALAYSIA

The 7 essential JDBC steps for Java database interaction

1. Load the Driver
2. Establish a connection
3. Create JDBC Statements
4. Execute SQL Statements
5. GET ResultSet
6. Close statement
7. Close connection

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Step 1: Load the Driver

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

Step 2 : Establish a connection

```
Connection conn =
DriverManager.getConnection("jdbc:mysql://localhost:3306/springdb",
"root", "");
```


UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Step 3: Create statement or preparedStatement

```
Statement stmt = Connection.createStatement();
```

Step 4 : Execute SQL Statement/PreparedStatement

Step 5 : Get ResultSet, and iterate through the resultset

```
ResultSet rs = stmt.executeQuery("SELECT * FROM PRODUCT")) {
        while (rs.next()) {
          //process the resultset
          }
```

Step 6 & 7 : Close Statement and Connection objects

```
stmt.close();
conn.close()
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

Updating Data Through Transactions

Transactions ensure the atomicity, consistency, isolation, and durability (ACID) properties of database operations

code example:

```
try {
    connection.setAutoCommit(false);
    //Perform multiple SQL operations as one atomic transaction unit
    //For example, update the quantity in the 'product' table for the
    //specified product ID and simultaneously add a new order to the
    //'Order' table
    connection.commit();
} catch (SQLException e) {
    connection.rollback();
} finally {
    connection.setAutoCommit(true);
}
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

Implementing Step 1 - Step 7 in ProductDAOJdbc.java

```java
//code segment for ProductDAOJdbc.java
@Service
public class ProductDAOJdbc {
    private static final String JDBC_URL = "jdbc:mysql://localhost:3306/springdb";
    private static final String USER = "root";
    private static final String PASSWORD = "";

    private Connection getConnection() {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            return DriverManager.getConnection(JDBC_URL, USER, PASSWORD);
        } catch (Exception ex) {
            ex.printStackTrace();
            throw new RuntimeException("Error establishing database connection", ex);
        }
    }
    public void add(Product product) {
        try (Connection connection = getConnection()) {
            String insertQuery = "INSERT INTO PRODUCT (name, price, quantity) VALUES (?, ?, ?)";
            try (PreparedStatement ps = connection.prepareStatement(insertQuery)) {
                ps.setString(1, product.getName());
                ps.setDouble(2, product.getPrice());
                ps.setInt(3, product.getQuantity());
                ps.executeUpdate();
                System.out.println("Product inserted successfully!");
            }
        } catch (Exception ex) {
            ex.printStackTrace();}
    }
```

Implementing Step 1 - 7 in ProductDAOJdbc.java (cont...)

```
44      //UPDATE OPERATION
45      public boolean update(String name) {
46          // complete the implementation here
47          return boolean(affectedRow!=0);
48      }
49
50      //DELETE OPERATION
51      public boolean delete(String name) {
52          // complete the implementation here
53          return boolean(affetedRow!=0);
54      }
55
56      //READ-SINGLE OPERATION
57      public Product getById(int id) {
58          Product product;
59          //complete the implementation here
60          return product;
61      }
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

Implementing Step 1 - 7 in ProductDAOJdbc.java (cont...)

```
56      //READ-SINGLE OPERATION
57      public Product getById(int id) {
58          Product product;
59          //complete the implementation here
60          return product;
61      }
62      //READ-ALL OPERATION
63      public List<Product> getAll() {
64          List<Product> productList = new ArrayList<>();
65
66          try (Connection connection = getConnection();
67               Statement stmt = connection.createStatement();
68               ResultSet rs = stmt.executeQuery("SELECT * FROM PRODUCT")) {
69
70              while (rs.next()) {
71                  Product product = new Product(rs.getString(2), rs.getDouble(3), rs.getInt(4));
72                  productList.add(product);
73              }
74          } catch (Exception ex) {
75              ex.printStackTrace();  }
76          return productList;
77      }
78  }// end class ProductDAOJdbc
```

ProductController invokes ProductDAOJdbc

```
17 @Controller
18 @RequestMapping("/product")
19 public class ProductController {
20     private final ProductDAOJdbc productDaoJdbc;
21
22     //dependency injection in constructor
23     public ProductController(ProductDAOJdbc productDaoJdbc) {
24         this.productDaoJdbc = productDaoJdbc;
25     }
26
27     @GetMapping("/tryCreate")
28     public ModelAndView tryCreate() {
29         Product p = new Product("Scalp Care Shampoo",25.50, 10);
30         productDaoJdbc.add(p);
31         ModelAndView mav = new ModelAndView("created");
32         mav.addObject("p",p);
33         return mav;
34     }
35
36     @GetMapping("/tryGetall")
37     public String tryGetall(Model model) {
38         List<Product> prodList = productDaoJdbc.getAll();
39         model.addAttribute("prodList",prodList);
40         return "allproduct";
41     }
42     //complete the remaining request patterns
43 }//end class ProductController
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

Using Relational Database in Web Application

Part 2:

Using Spring JDBC Template - focuses on the Spring JDBC Template, a higher-level abstraction that simplifies database operations within the Spring framework.

OVERVIEW

Spring JDBC allows developers to focus on core logic. Spring handles all the low level details and repetitive tasks like opening and closing of connections, exceptions, iteration of resultsets, transactions etc.

UTM UNIVERSITI TEKNOLOGI MALAYSIA

SPRING JDBC – WHO DOES WHAT?

Following table shows what actions spring take care of and which actions are responsibility of application developer

Action	Spring	You
Define connection parameters.		X
Open the connection.	X	
Specify the SQL statement.		X
Declare parameters and provide parameter values		X
Prepare and execute the statement.	X	
Set up the loop to iterate through the results (if any).	X	
Do the work for each iteration.		X
Process any exception.	X	
Handle transactions.	X	
Close the connection, statement and resultset.	X	

UTM UNIVERSITI TEKNOLOGI MALAYSIA

cont...

Step 1: add dependency in pom.xml :

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-jdbc</artifactId>
    <version>5.2.25.RELEASE</version>
</dependency>
```

Step 2: Create a configuration class, to set up the **DataSource** and **JdbcTemplate**

The **DatabaseConfig.java** is annotated with ***@Configuration***, indicating that it contains **bean** definitions. It defines **two beans**:

1. dataSource(): Configures a DriverManagerDataSource bean, which is a basic DataSource implementation provided by Spring.

2. jdbcTemplate(DataSource dataSource): Configures a JdbcTemplate bean, injecting the DataSource bean created in the previous step. Then, this JdbcTemplate bean can be used to execute the SQL queries to the database.

UTM UNIVERSITI TEKNOLOGI MALAYSIA

```java
//partial code for DatabaseConfig.java
package service;
//import statements here

@Configuration
public class DatabaseConfig {
        private DriverManagerDataSource dataSource;

    @Bean
    public DataSource dataSource() {
        this.dataSource = new DriverManagerDataSource();
        //in real scenario, put the following hard-coded values into file ie, jdbcConfig.properties
        dataSource.setDriverClassName("com.mysql.cj.jdbc.Driver");
        dataSource.setUrl("jdbc:mysql://localhost:3306/springdb");
        dataSource.setUsername("root");
        dataSource.setPassword("");
        return dataSource;
    }

    @Bean
    @Autowired      //dependency injection using constructor
    public JdbcTemplate jdbcTemplate(DataSource dataSource) {
        return new JdbcTemplate(dataSource);
    }
    //you may add code for close() and cleanUp() to properly close connections (omitted for brevity)
} //end class DatabaseConfig
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

cont...

Step 3: use the instance of JdbcTemplate, to invoke related methods for executing queries

class: JdbcTemplate (org.springframework.jdbc.core.JdbcTemplate): The core class of Spring JDBC framework, simplifies the use of JDBC and helps avoid common errors. It provides various methods for executing SQL queries, updates, and other operations.

important methods:

1. query: public <T> List<T> query(String sql, RowMapper<T> rowMapper, Object... args) throws DataAccessException

Executes a query and maps each row to an object using the provided RowMapper. Returns a list of objects.

example:

```
String sql = "select name, address, contactNum, email from customer";
List<Customer> custList = jdbcTemplate.query(sql, new BeanPropertyRowMapper(Customer.class));
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

cont...

important methods (cont...):

2. update: public int update(String sql, Object... args) throws DataAccessException

Executes an SQL update, insert, or delete statement. Returns the number of affected rows.

example:

```
String sql = "INSERT INTO CUSTOMER (name, address, contactNum, email) values (?,?,?,?)";
Object args[] = {"ali","taman","1234567","email@gmail.com"};
jdbcTemplate.update(sql,args);
```

3. queryForObject:

public <T> T queryForObject(String sql, Class<T> requiredType, Object... args) throws DataAccessException

Executes a query and returns a single result as an object of the specified type.

example:

```
String sql = "SELECT * FROM CUSTOMER WHERE ID=?";
Customer cust = jdbcTemplate.queryForObject(sql,
        new BeanPropertyRowMapper<Customer>(Customer.class), id);
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

cont...

important methods (cont...):

4. queryForList:

public List<Map<String, Object>> queryForList(String sql, Object... args) throws DataAccessException

Executes a query and returns the results as a list of maps, where each map represents a row with column names and values.

5. batchUpdate

public int[] batchUpdate(String sql, List<Object[]> batchArgs) throws DataAccessException

Executes a batch of SQL updates. Takes a list of parameter arrays, each representing the parameters for a single update.

UTM UNIVERSITI TEKNOLOGI MALAYSIA

cont...

class BeanPropertyRowMapper : org.springframework.jdbc.core.BeanPropertyRowMapper
Maps rows to objects by matching the column names to the properties of the target class. It simplifies the mapping process, especially when the column names in the ResultSet match the field names in the Java class.

example use:

```
String sql = "select name, address, contactNum, email from customer";
List<Customer> custList = jdbcTemplate.query(sql, new BeanPropertyRowMapper(Customer.class));
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

```
// partial code for CustomerDAO.java
//import statements here

@Service
public class CustomerDAO {
   private JdbcTemplate jdbcTemplate;

   @Autowired       //dependency injection using constructor
   public CustomerDAO(JdbcTemplate jdbcTemplate) {
      this.jdbcTemplate = jdbcTemplate;  //reference to bean instance in IoC container
   }
   //IMPLEMENTATION OF CRUD OPERATIONS HERE
   //CREATE
   public String add() {
            String sql = "INSERT INTO CUSTOMER (name, address, contactNum, email) values (?,?,?,?)";
            Object args[] = {"ali","taman","1234567","email@gmail.com"};
            jdbcTemplate.update(sql,args);
            return "success";
   }
   //READ-ALL
   public List<Customer> getall(){
            String sql = "select name, address, contactNum, email from customer";
            List<Customer> custList = jdbcTemplate.query(sql, new BeanPropertyRowMapper(Customer.class));
            return custList;
   }//end getall
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

```
//cont....
//READ ie BY-NAME (this may return multiple rows)
public List<Customer> getByName(String name) {

            String sql = "SELECT * FROM CUSTOMER WHERE NAME=?";
            List<Customer> custList = jdbcTemplate.query(sql,
                  new    BeanPropertyRowMapper <Customer>(Customer.class), name);
            return custList;
      } //end getByName

    //impl more READ methods ie READ-BY-ID (this return single row)

   //impl UPDATE ie BY-ID

   //impl DELETE ie BY-ID

}//end CustomerDAO
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

Step 4: Invoke the CustomerDAO from the Controller class

```
19 //code for CustomerController.java
20 @Controller
21 @RequestMapping("/customer")
22 public class CustomerController {
23
24     @Autowired
25     private CustomerDAO custDao;
26
27     public CustomerController() {
28     }
29
30     @GetMapping("/getall")
31     public ModelAndView getall() {
32         List<Customer> custList = custDao.getall();
33         ModelAndView mav = new ModelAndView("customerList");
34         mav.addObject("custList",custList);
35     return mav;
36     }
37
38     @ResponseBody
39     @GetMapping("/getById")
40     public String getById(@RequestParam("id") int id, Model model) {
41         Customer cust = custDao.getById(id);
42         model.addAttribute("cust",cust);
43         return  "viewCustById";
44     }
45     //more codes here
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

Using Relational Database in Web Application

Part 3:
Using Hibernate - an ORM framework.
Object/Relational Mapping (ORM) refers to the technique of mapping a data representation from an object model to a relational data model with a SQL-based schema

Hibernate is an open source java based library framework, is a very powerful ORM solution built on top of JDBC (Java Database Connectivity) API.

Hibernate Architecture Components

:Hibernate follows a layered architecture comprising three main components: **Configuration, SessionFactory**, and **Session**

1. Configuration:

The Configuration component is responsible for configuring Hibernate and creating a representation of the application's domain model.

Role:

Sets up Hibernate by reading configuration files, mapping documents, and creating a configuration instance.

2. SessionFactory:

The SessionFactory is a factory of session objects and is a heavyweight object.

Role:

Acts as a factory for creating Session instances. It is thread-safe and should be instantiated only once in the application.

Responsibilities:

Manages Hibernate's configuration information.

Provides a convenient mechanism for opening sessions.

UTM UNIVERSITI TEKNOLOGI MALAYSIA

3. Session:

Session is a lightweight object that represents a single-threaded unit of work.

Role:

Represents a connection to the database and provides CRUD operations. It is created by the SessionFactory.

Responsibilities:

Wraps a JDBC connection.

Performs database operations (save, update, delete, query).

UTM UNIVERSITI TEKNOLOGI MALAYSIA

Roles and Relationships

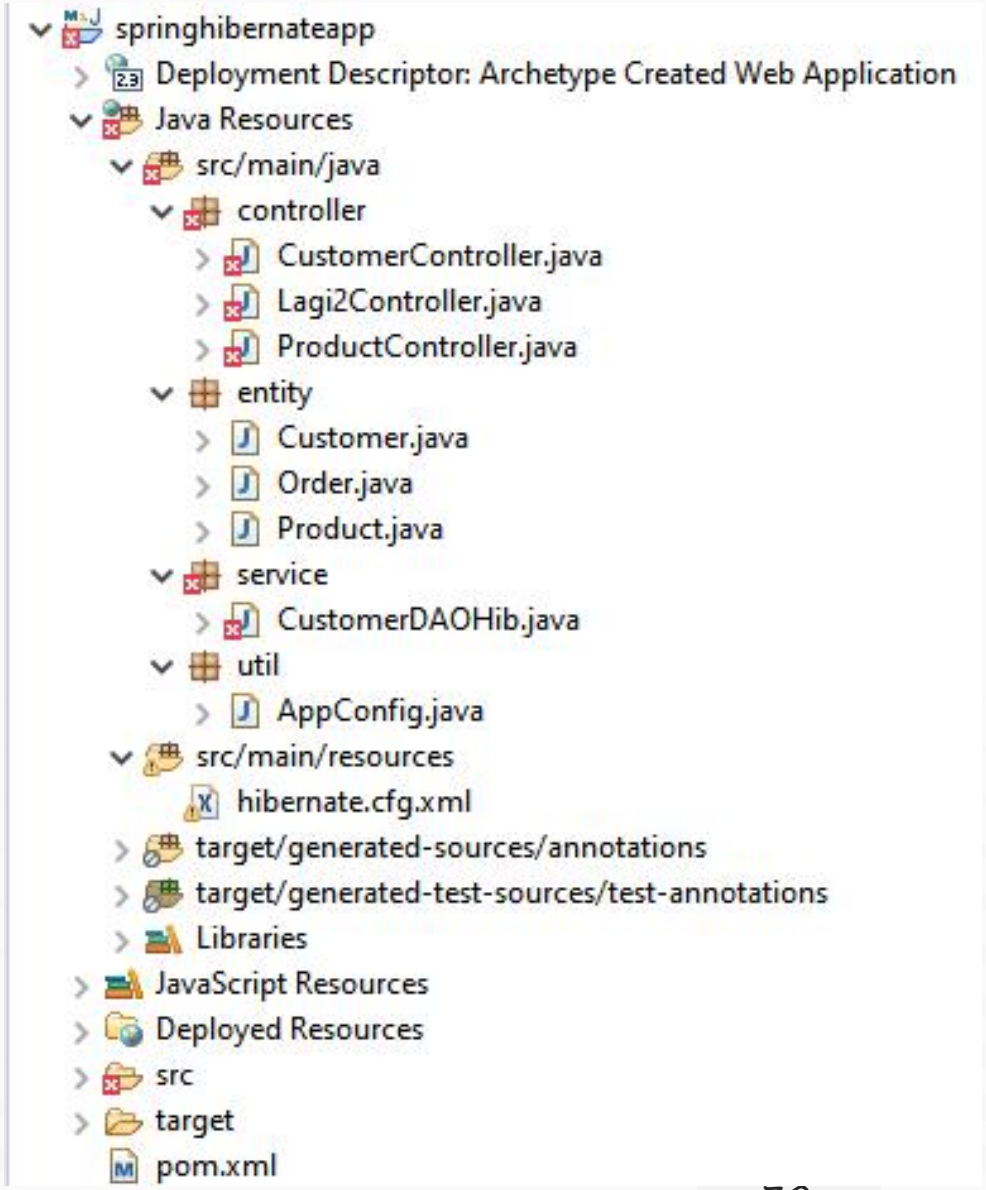
Configuration: Prepares Hibernate for use by setting up the environment and mapping files.
SessionFactory: Acts as a factory to create Session instances, manages configuration, and maintains a cache of data, optimizing performance.
Session: Represents a single unit of work and provides methods for interacting with the database. Manages transactional behavior.

Interaction Flow:

Configuration: Initializes Hibernate settings.
SessionFactory: Creates a Session based on the configured settings.
Session: Performs database operations and manages the transactional context.

UTM UNIVERSITI TEKNOLOGI MALAYSIA

project structure



Hibernate ConfigurationFile: (src/main/resources/hibernate.cfg.xml)

```
<hibernate-configuration>
    <session-factory>
        <!-- Database connection settings -->
        <property name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
        <property name="hibernate.connection.url">jdbc:mysql://localhost:3306/springdb</property>
        <property name="hibernate.connection.username">root</property>
        <property name="hibernate.connection.password"></property>

        <!-- Hibernate C3P0 connection pool settings -->
        <property name="hibernate.c3p0.min_size">5</property>
        <property name="hibernate.c3p0.max_size">20</property>
        <property name="hibernate.c3p0.timeout">300</property>
        <property name="hibernate.c3p0.max_statements">50</property>
        <property name="hibernate.c3p0.idle_test_period">3000</property>

        <!-- Other Hibernate properties -->
        <property name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>
        <property name="hibernate.show_sql">true</property>
        <property name="hibernate.hbm2ddl.auto">update</property>

        <!-- Mention annotated classes -->
        <mapping class="entity.Customer"/>
        <mapping class="entity.Product"/>
        <mapping class="entity.Order"/>
        <mapping class="entity.Invoice"/>
    </session-factory>
</hibernate-configuration>
```

UTM UNIVERSITI TEKNOLOGI MALAYSIA

File: (java/source/util/ApplicationConfig.java)

```java
@Configuration
@EnableTransactionManagement
public class AppConfig {

    @Bean
    public HibernateTransactionManager transactionManager(SessionFactory sessionFactory) {
        HibernateTransactionManager transactionManager = new HibernateTransactionManager();
        transactionManager.setSessionFactory(sessionFactory);
        return transactionManager;
    }

    @Bean
    public LocalSessionFactoryBean sessionFactory(DataSource dataSource) {
            LocalSessionFactoryBean sessionFactory = new LocalSessionFactoryBean();
            sessionFactory.setDataSource(dataSource);
            sessionFactory.setPackagesToScan("entity");
            // Set other necessary properties like hibernateProperties, annotatedClasses, etc.
            return sessionFactory;
        }    // Other beans can be defined here...
}
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

file: (java/service/CustomerDAOHib.java)

```java
@Service
public class CustomerDAOHib {

	@Autowired
	private SessionFactory sessionFactory;

	// Method to retrieve all Customer entities from the database
	@Transactional
	public List<Customer> getall() {
		// Obtain the current Hibernate session
		Session currentSession = sessionFactory.getCurrentSession();
		// Create an HQL query to select all CustomerVerHibernate entities
		String hql = "FROM Customer";
		Query<Customer> query = currentSession.createQuery(hql, Customer.class);
		try {
			// Execute the query and return the result
			List<Customer> customers = query.list();
			return customers;
		} catch (Exception e) {
			// Handle or log the exception
			throw new RuntimeException("Error fetching customers", e);
		}
	}
}
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

Invoke the CustomerDAOHib from the Controller class

```
@Controller
@RequestMapping("/lagi2customer")
public class Lagi2Controller {

        @Autowired
        private CustomerDAOHib custDaoHib;

        @GetMapping("/getall")
        public ModelAndView getall() {
                List<Customer> custList = custDaoHib.getall();
                ModelAndView mav = new ModelAndView("customerList");
                mav.addObject("custList",custList);
        return mav;
        }
```


UTM UNIVERSITI TEKNOLOGI MALAYSIA

file: (java/entity/Customer.java)

```java
@Entity
@Table(name = "Customer")
public class Customer{

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String address;
    private String contactNum;
    private String email;

    // Constructors, getters, and setters
```

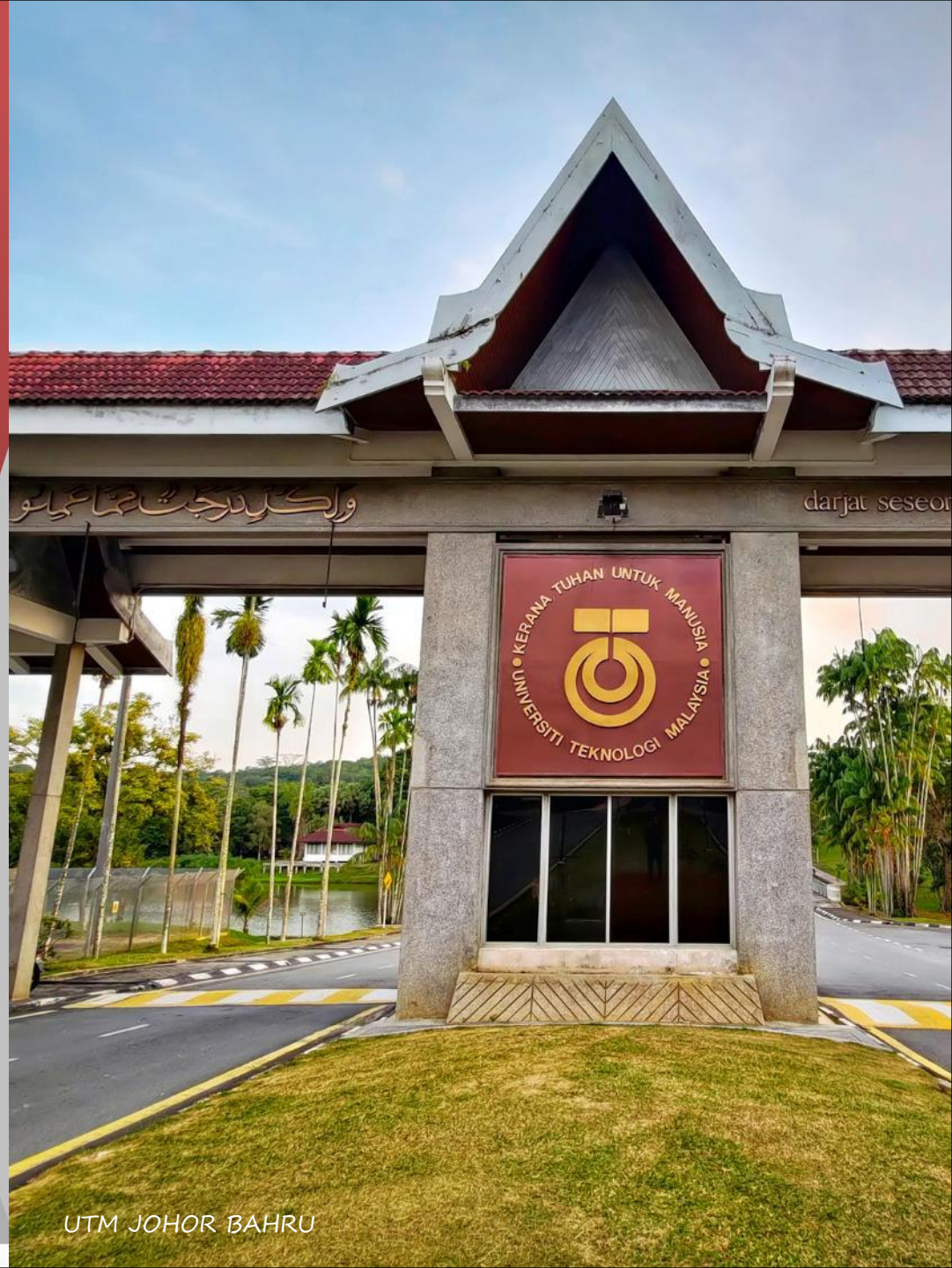

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

TOPIC 8 – Using Relational Database (MySQL)

The End

innovative • entrepreneurial • global



UTM JOHOR BAHRU